Assignment 2: HTTP APIs, gRPC and Metrics

Inora jewelry store, Enterprise Software Development, Fall 2026

Maaz (individual assignment)

Repository: https://github.com/maazahmedsiddiqui/inora-observability-part2

1. What was built

Assignment 1 had a small Flask app with `GET /catalog`, `POST /order` (failed randomly 20% of the time, no input, no order id) and `POST /cancel` (no order id). For Assignment 2 I chose the user task **"place an order"** and rebuilt it:

- **Part A:** a REST-style HTTP API with validation, one error format, an OpenAPI spec, idempotency, rate limiting, and a Python caller with timeout, retry and circuit breaker.
- **Part B:** the operation `PlaceOrder` as a gRPC service in its own container. The HTTP version stays. Both call the same business module (`inora_core.Store`) and the same SQLite file on a shared Docker volume. The gRPC handler is not a wrapper around the HTTP endpoint.
- **Part C:** audit of the A1 metrics, new gRPC metrics scraped by Prometheus, and a Grafana dashboard showing gRPC beside HTTP.

All commands to run and test everything are in `README.md`. The suite has 95 automated tests.

2. Part A: HTTP API

2.1 The six REST constraints

Constraint	Plain-language meaning	Example in this API
Client-server	UI/caller and server are separate and evolve independently	The Python caller and Swagger UI call the API only over HTTP; neither shares code with it
Stateless	Every request carries all the information needed; the server keeps no session between requests	`POST /orders` carries product, quantity, payment token and `Idempotency-Key` each time; no login session
Cacheable	Responses say whether they may be reused	Not demonstrated: the API sets no cache headers (order data changes and is not meant to be cached)
Uniform interface	Same conventions everywhere: resources named by URL, standard methods and status codes, self-describing messages	Nouns as paths (`/products`, `/orders/{id}`), `GET`/`POST`, `201` + `Location`, errors as `application/problem+json`
Layered system	The caller cannot tell whether it talks to the server directly or to an intermediary	Not demonstrated by design; the API reads `X-Forwarded-For`, so it can sit behind a proxy
Code on demand (optional)	Server may send executable code to the client	Not demonstrated (optional)

2.2 Endpoints reviewed

Method and path	Purpose	Success	Errors
`GET /products`	List products	200	none
`GET /products/{product_id}`	One product	200	404
`POST /orders`	Place an order (the user task)	201 (200 on replay)	400, 404, 409, 422, 429, 500, 503
`GET /orders/{order_id}`	Read an order	200	404
`POST /orders/{order_id}/cancellations`	Cancel an order	200	404

`/health` and `/metrics` are excluded as the manual says. Input rules for `POST /orders`: `product_id` must exist, `quantity` 1 to 10, `payment_token` must look like `tok_...`, `Idempotency-Key` 1 to 128 characters. Errors always use RFC 9457 `application/problem+json` with `type`, `title`, `status`, `detail`, `code`, `trace_id`.

Business error	HTTP status	gRPC status
validation error	400 Bad Request	INVALID_ARGUMENT
unknown product or order	404 Not Found	NOT_FOUND
payment declined (`tok_declined`)	422 Unprocessable Content	FAILED_PRECONDITION
out of stock	409 Conflict	RESOURCE_EXHAUSTED
idempotency key reused with a different request	409 Conflict	ALREADY_EXISTS
rate limit exceeded	429 + `Retry-After`	not applied to gRPC

2.3 Before and after: “place an order”

	Details
Old endpoint	`POST /order`
Problem	No input, so nothing to validate and no order id returned. Failed randomly 20% of the time, so failures were not repeatable and not tied to the request. A retry could not be made safe. No rate limit. Nothing shared with another service.
Change	New endpoint `POST /orders` with a JSON body, validation, the order stored in SQLite, `201` + `Location`, deterministic failures (`tok_declined` gives 422, an unavailable quantity gives 409), `Idempotency-Key`, per-IP rate limit, problem+json errors and `traceparent` correlation.
Reason	Placing an order is the most important customer action. A client needs an id to refer to the order later, and a retry after a timeout must not create two orders.

Other endpoints:

Before	After	Change
`GET /catalog`	`GET /products`, `GET /products/{product_id}`	Renamed to a resource noun; added single-product read
`POST /cancel` (no id)	`POST /orders/{order_id}/cancellations`	Cancelling is modelled as a sub-resource of one order; repeating a cancel does not cancel twice
(none)	`GET /orders/{order_id}`	New, so a client can read what it created
`/health`, `/metrics`	unchanged	Left alone

Callers updated: `traffic.py` now calls `/products`, `/orders` and `/orders/{id}/cancellations`; the new `caller.py` and `generate_traffic.py` use the new routes.

2.4 OpenAPI

What it is. OpenAPI is a machine-readable, language-neutral description of an HTTP API: its paths, methods, inputs, and the success and error responses. Tools use the same file to generate interactive documentation, client code and mock servers, and to check that a server really behaves as documented, so documentation and code cannot silently drift apart.

The spec (`openapi.yaml`, OpenAPI 3.0.3) describes all five endpoints, their parameters, the request body, every response status and the problem+json error schema. It is served by the API at `/openapi.yaml`.

How it was used.

1. *Inspect and test live:* a Swagger UI container (`http://localhost:8080`) loads the spec. Figures 1 to 4 show calls made from it.
2. *Contract tests:* `tests/test_openapi_contract.py` (18 tests) reads the same file and checks the running app against it: routes and methods match, every response has a documented status, content type and schema-valid body, declared headers are present, inputs the schema rejects get 400, and every documented outcome of `POST /orders` (201, 200, 400, 404, 409, 422, 429, 503) actually occurs. All 18 pass (`evidence/openapi_contract_tests.txt`). The tests can fail: I broke a scratch copy of the spec three ways (missing 422, wrong `maximum`, renamed required field) and each break was caught. Writing the tests found two gaps in the first spec (no 500 response, no `X-RateLimit-*` headers on 429), now fixed.

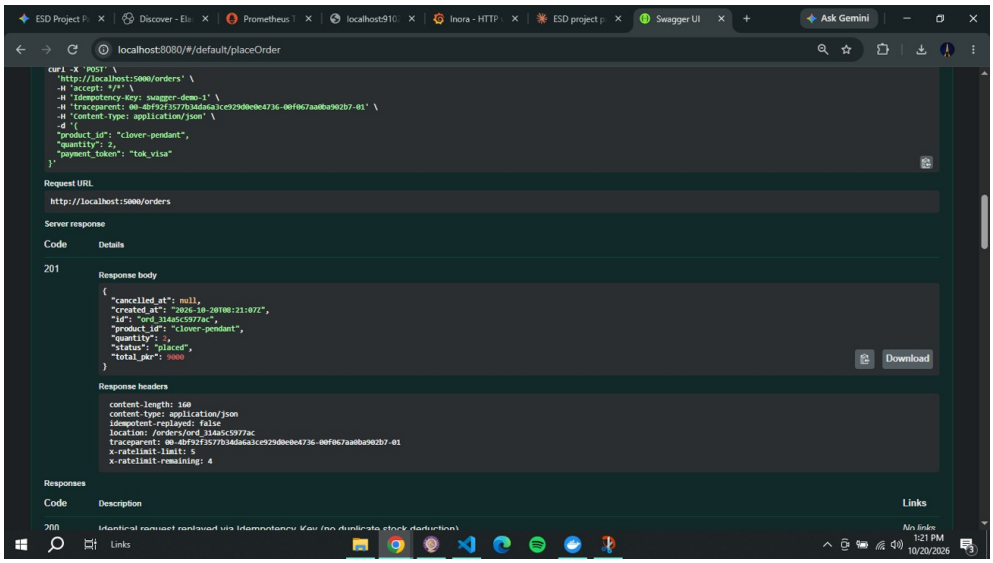


Figure 1: Swagger UI, POST /orders, first call with key swagger-demo-1: 201 Created, Location header, idempotent-replayed: false

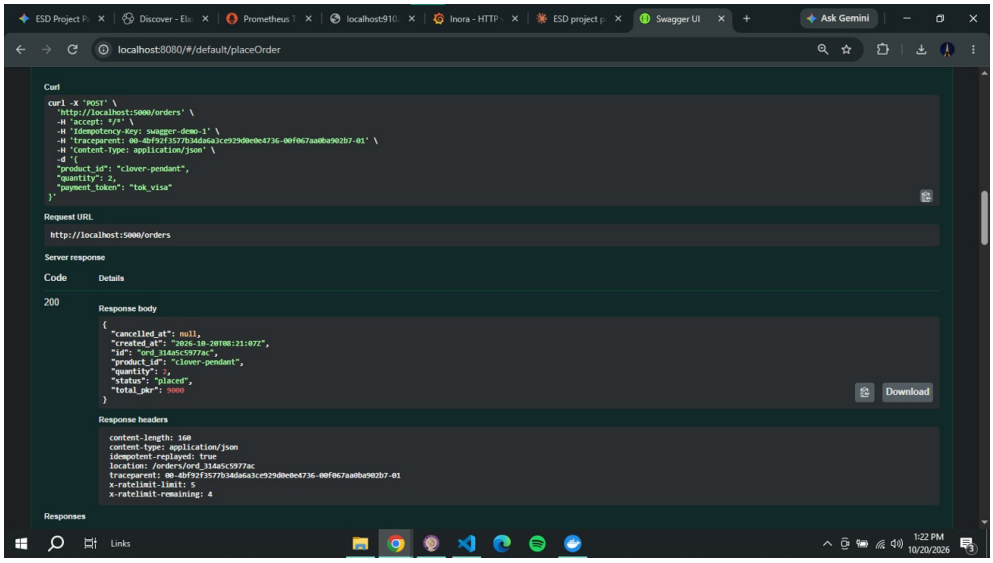


Figure 2: Swagger UI, same request and key again: 200 OK, idempotent-replayed: true, same order id (no duplicate)

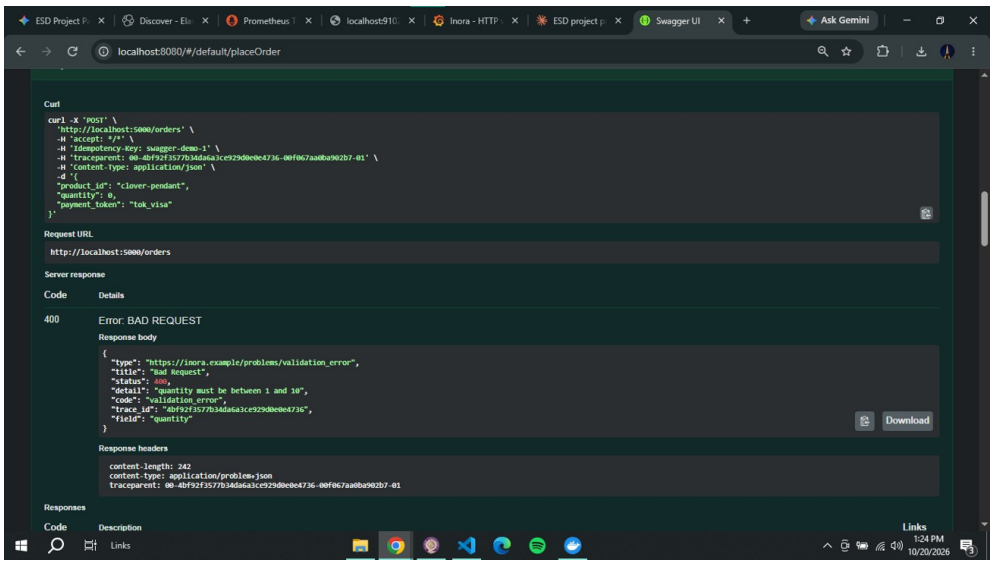


Figure 3: Swagger UI, quantity 0: 400 Bad Request, application/problem+json, field: quantity

2.5 Idempotency

`POST /orders` accepts an `Idempotency-Key`. The server stores the key with a hash of the request in the same SQLite transaction that creates the order (`BEGIN IMMEDIATE`, so two requests cannot race, even from two containers). Same key and same body: the original order is returned with `200` and `Idempotent-Replayed: true` and stock is not deducted again (Figure 2). Same key and a different body: `409` (`idempotency_conflict`). `GET` is naturally safe to repeat, and a repeated cancel is harmless: the order is only cancelled and its stock restored once.

2.6 Rate limiting

- **Endpoint:** `POST /orders` only.
- **Limit:** 5 requests per 10-second fixed window.
- **Who shares it:** all requests from the same client IP address (taken from `X-Forwarded-For`, otherwise the socket address). Different IPs have separate allowances. Single-instance, in-memory implementation, as the manual allows. Trusting `X-Forwarded-For` is a known simplification: a real deployment would only trust it from its own proxy.

- **When allowed again:** when the current window ends. `429` carries `Retry-After` (seconds until then) and `X-RateLimit-Limit` / `X-RateLimit-Remaining`.

Result from `evidence/caller_ratelimit.txt` (7 orders in a row):

Order	Result
1 to 5	`HTTP 201` (accepted)
6, 7	`HTTP 429`, `Retry-After = 8 s`, not treated as a failure and not retried
after waiting the advertised 8 s	order 8 gives `HTTP 201`

2.7 Tests: valid, invalid, repeated

File	Tests	Covers
`test_core.py`	17	business rules, idempotency, stock, cancel
`test_http_api.py`	19	valid order (201), invalid input (400/404/422), repeated request (200 replay, 409 conflict), rate limit, fault switches
`test_caller.py`	16	timeout, retry rules, backoff, breaker states
`test_grpc.py`	17	success and every failure status, deadline, trace metadata, shared store with HTTP
`test_grpc_metrics.py`	8	metric names, labels, counts
`test_openapi_contract.py`	18	spec against the running app
Total	**95**	all pass (`python -m pytest -q`)

2.8 Python caller: timeout, retry, circuit breaker

`caller.py` acts like another service calling `POST /orders`.

Setting	Value
Timeout	2.0 s per attempt
Attempts	3 (initial try plus 2 retries)
Backoff	1.0 s, doubling (1 s, 2 s, capped at 4 s), +/-10% jitter
Retried only on	timeout, connection error, HTTP 500/502/503/504. Never on 4xx
Rate limit (429)	reported separately; honours `Retry-After`; not a failure for the breaker
Retry safety	a POST is retried only because it carries an `Idempotency-Key`; the same key and trace id are reused on every attempt
Breaker	opens after 3 consecutive failures; recovery period 5 s; then one half-open trial call

The failure test uses a slow server and 503 errors (switched on with the `X-Simulate-Fault` header), not rate limiting. Output from `evidence/caller_reliability.txt`:

Call	What happened
1	Normal: attempt 1, `201` in 0.03 s, breaker closed
2	Slow server: attempt 1 timed out after 2.0 s (failure 1), retry in 0.95 s with the same key, attempt 2 `200 OK (replay)` in 0.17 s. The server had finished the first attempt; the key stopped a duplicate order
3	Server error: attempt 1 `503`, retry in 1.01 s, attempt 2 `201`
4	Server down: `503` on all 3 attempts (retries after 1.02 s and 2.02 s), breaker opens on the 3rd failure, caller gives up

Call	What happened
5, 6	**Breaker open:** blocked immediately, no request sent (trial allowed in 5.0 s)
wait 5.2 s	recovery period
7	**Recovery:** half-open trial call `201` in 0.02 s, breaker closed
8	Normal call `201`, breaker closed

Summary line: “all 8 calls behaved as designed”.

3. Part B: gRPC service

3.1 Contract and code

- `proto/inora/v1/inora.proto`: service `OrderService`, unary `rpc PlaceOrder(PlaceOrderRequest) returns (PlaceOrderResponse)`. Request: `product_id`, `quantity`, `payment_token`, `idempotency_key`. Response: `Order` and `replayed`.
- **Generation command** (from the project root): `python -m grpc_tools.protoc -I proto --python_out=. --grpc_python_out=. proto/inora/v1/inora.proto` The generated `inora_pb2.py` and `inora_pb2_grpc.py` are committed.
- **Handler:** `grpc_server.py`, `OrderService.PlaceOrder`. It reads the trace id from metadata, calls `Store.place_order(...)` (the same function the HTTP route calls), translates a `DomainError` into a gRPC status, and sets trailing metadata `error-code` and `traceparent`.
- **Client:** `grpc_client.py` takes product, quantity, token and key on the command line, sets a deadline (`--deadline`, default 5 s) and prints the order or the status, domain code and trace id.
- **Container:** `Dockerfile.grpc`, service `grpc` in `docker-compose.yml`, port 50051 (metrics on 9102). It shares the `inora-data` volume with `api`.

3.2 Call paths

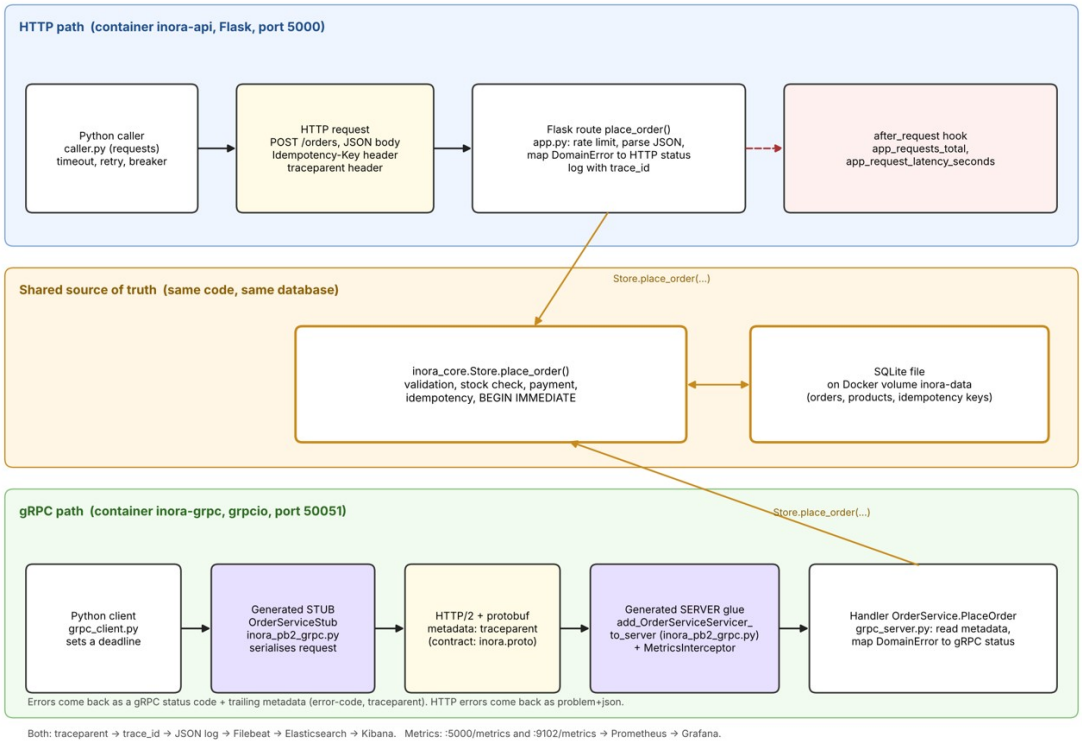


Figure 4: Call paths. HTTP: caller, Flask route, shared Store. gRPC: client, generated stub, HTTP/2, generated server glue and interceptor, handler, shared Store. Both end at the same SQLite file.

HTTP path: the caller sends `POST /orders`; the Flask route applies the rate limit, parses JSON, calls `Store.place_order`, and maps any `DomainError` to a status. gRPC path: the client calls `OrderServiceStub.PlaceOrder`; the generated stub serialises the request to protobuf and sends it over HTTP/2 with `traceparent` in metadata; the generated server glue (`add_OrderServiceServicer_to_server`) passes it through the `MetricsInterceptor` to the handler, which calls the same `Store.place_order`.

3.3 One success and one failure through both versions

From `evidence/part_b_http_vs_grpc.txt`:

Case	HTTP	gRPC	Meaning
Success	`201 Created`, order `ord_a915e5c11417`	`OK`, order `ord_b61733de1d52`	Order placed; each call created its own order, trace `8239755bb362295b4096dc06045386a5`
Failure: payment declined (`tok_declined`)	`422`, code `payment_declined`	`FAILED_PRECONDITION`, `payment_declined`	The request was valid but the payment cannot be processed. Trace `f8368bf75aedf10795606dc52037cbe5`
Shared source of truth	HTTP placed `ord_9b872de49aac`	gRPC with the same key returned `ord_9b872de49aac` (replayed)	The gRPC call found the order the HTTP call wrote

Other statuses use the same table as section 2.2. A client deadline appears on the server as `CANCELLED`, not `DEADLINE_EXCEEDED` (the server sees the client leave), and the handler still finishes the work, which is why an idempotency key makes a retry safe.

3.4 A1 logs and trace correlation across the boundary

Both services write the A1 JSON log format (`time`, `service_name`, `severity`, `message`, `request_id`, `trace_id`, `endpoint`, `status_code`) to files that Filebeat ships to Elasticsearch. A W3C `traceparent` travels as an HTTP header or gRPC metadata and becomes `trace_id` in the logs. Searching Kibana for `trace_id : "f8368bf75aedf10795606dc52037cbe5"` finds both log lines of the declined payment, one from `inora-http-api` (12:03:46.311) and one from `inora-grpc` (12:03:47.912) (Figure 5).

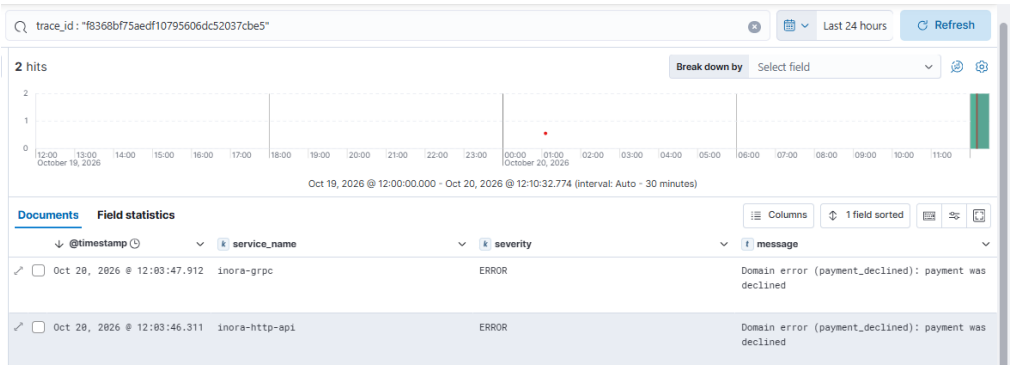


Figure 5: Kibana, one trace id, 2 hits: inora-http-api and inora-grpc

4. Part C: Metrics

4.1 Audit of the A1 metrics and the gaps

A1 metric	Useful?	Gap	Fix
`app_requests_total{method,endpoint,http_status}`	Yes	Hand-incremented, so `/cancel` and error paths were missed	Counted once per request in `after_request`, using the route template (`/orders/<order_id>`), so ids never become labels
`app_active_requests`	Yes	Only two routes tracked	`before_request`/ `after_request` for all routes except `/metrics`
`app_request_latency_seconds`	Partly	`/catalog` only, no labels, default buckets	Labels `method`, `endpoint`; buckets 5 ms to 5 s
`app_order_processing_seconds` (summary)	Partly	No quantiles, cannot be aggregated	Kept for the average; percentiles come from histograms
`business_orders_total{status}`	Yes	`failed` was a random 20% fault; gRPC orders invisible	`failed` = declined payment or out of stock; gRPC exports the same metric
rate limiting	Missing	No visibility	`http_rate_limit_decisions_total{decision}`
gRPC	Missing	No metrics	`grpc_server_*` (below)
stock	Missing	The one number both front doors change	`inora_product_stock{product_id}`

4.2 New metrics

All gRPC duration is measured **on the server**, inside the handler. It excludes network time, serialisation and waiting for a worker thread. It is not a client-side measurement. All labels are bounded; no user, request, order, key or trace id is ever a label. Prometheus job `inora_grpc` scrapes `grpc:9102` every 5 s (Figure 6).

Metric	Purpose	Type	Unit	Labels	Code location
`grpc_server_handled_total`	Calls and error rate by method and status	Counter	calls	`grpc_service`, `grpc_method`, `grpc_code`	`grpc_metrics.py`, `MetricsInterceptor`
`grpc_server_handling_seconds`	Handler duration, p50 and p95	Histogram (buckets 5 ms to 5 s)	seconds	`grpc_service`, `grpc_method`	same interceptor
`grpc_server_in_flight`	Calls being handled now (saturation of the 10-worker pool)	Gauge	calls	`grpc_service`, `grpc_method`	same interceptor
`business_orders_total` (gRPC copy)	Orders by outcome over gRPC; same meaning as the HTTP counter so both can be summed	Counter	orders	`status` = success or failed	`grpc_server.py`, `PlaceOrder`
`inora_product_stock`	Units in stock per product, read from the shared DB at scrape time	Gauge (custom collector)	units	`product_id` (4 values)	`grpc_metrics.py`, `StockCollector`

Queries (as used in the dashboard panels):

```
Calls/s by method and status (1 m):
sum by (grpc_method, grpc_code)
(rate(grpc_server_handled_total{grpc_method!="unknown"}[1m]))

Error rate by method (5 m):
sum by (grpc_method) (rate(grpc_server_handled_total{grpc_code!="OK",
grpc_method!="unknown"}[5m]))
  / sum by (grpc_method) (rate(grpc_server_handled_total{grpc_method!
="unknown"}[5m]))

p95 handling time (5 m)  (p50: use 0.50):
histogram_quantile(0.95, sum by (le)
(rate(grpc_server_handling_seconds_bucket{grpc_method="PlaceOrder"}[5m])))

In-flight:
sum(grpc_server_in_flight)

Orders per minute by outcome, both front doors:
sum by (status) (rate(business_orders_total[1m])) * 60

Stock:
inora_product_stock
```

HTTP side used for comparison: `app_requests_total`, `app_request_latency_seconds_bucket`, `app_active_requests`, `http_rate_limit_decisions_total`.

Time windows. Rates use 1 minute. Error rate, p50 and p95 use 5 minutes, about 60 scrapes at 5 s, enough samples for a stable quantile at demo traffic.

Buckets. One SQLite commit takes a few milliseconds, a busy laptop tens to hundreds; the caller's timeout is 2 s and the gRPC default deadline 5 s. So 5 ms to 5 s resolves the normal range and still shows the tail. The HTTP histogram uses the same edges, so the two p95 panels compare directly.

In-flight, not message size. `PlaceOrder` messages are a few dozen bytes of fixed shape, so their size never changes and says nothing about health. In-flight shows how close the server is to its worker limit and mirrors the HTTP `app_active_requests` gauge.

Another missing metric. `inora_product_stock`: stock is the one state both front doors change, so it is direct evidence of the shared source of truth, and it lets the shop spot "limited-necklace = 0" before customers see out-of-stock errors.

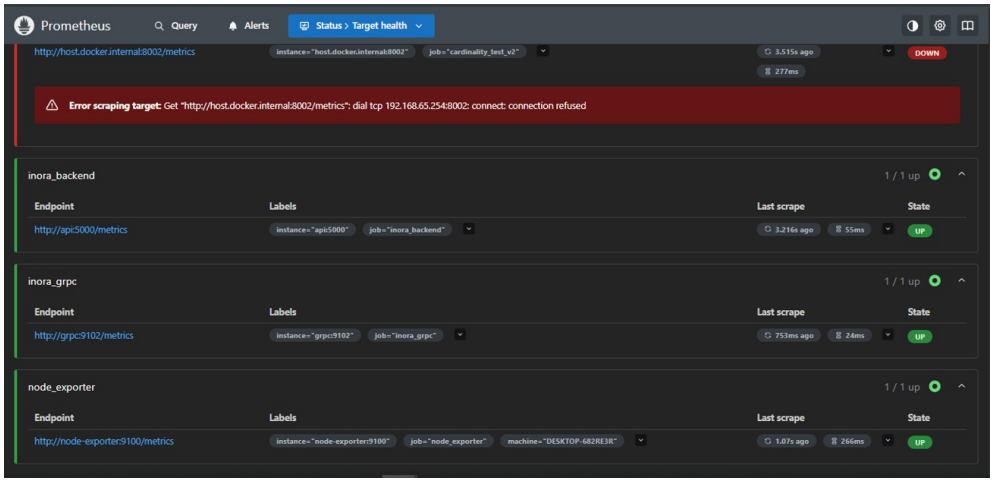


Figure 6: Prometheus targets: inora_backend (api:5000), inora_grpc (grpc:9102) and node_exporter are UP. The red cardinality_test job is a leftover Assignment 1 target for a script that is not running.

4.3 Repeatable traffic

`python generate_traffic.py` sends a fixed 11-case sequence per round, one HTTP call and one gRPC call each, every 2.5 s: 6 success, 2 declined payment, 1 invalid input, 1 unknown product, 1 limited necklace (the last unit, then out of stock). 44 rounds gave (`evidence/part_c_traffic.txt`):

	201/OK	422/ FAILED_PRECONDITION	400/ INVALID_ARGUMENT	404/NOT_FOUND	409/ RESOURCE_EXHAUSTED
HTTP	25	8	4	4	3
gRPC	24	8	4	4	4

The difference is in the stock case. In round 11 the HTTP call took the last necklace (201) and the gRPC call, arriving next, got RESOURCE_EXHAUSTED. That is the shared database working as intended, not a bug.

4.4 Dashboard evidence

Dashboard `Inora - HTTP vs gRPC (Assignment 2)` is provisioned from `grafana/dashboards/inora-http-vs-grpc.json` (11 panels; data source provisioned in `grafana/provisioning`).

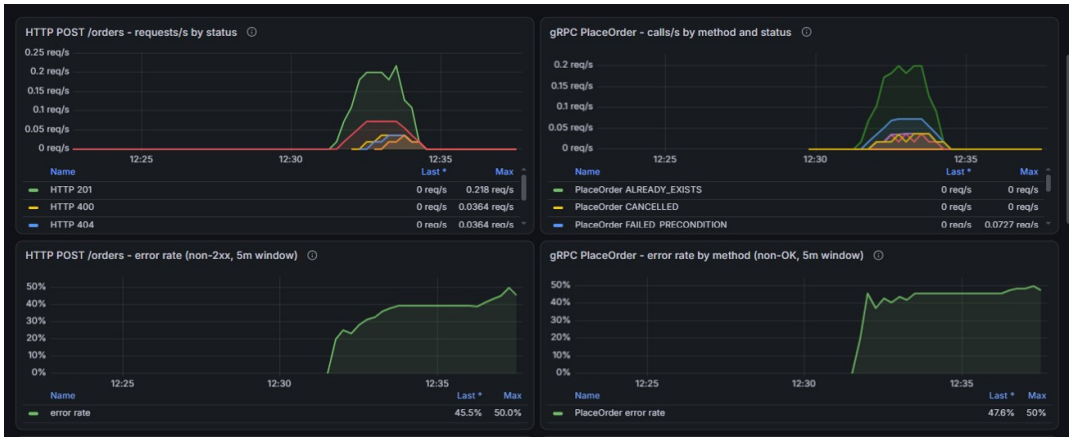


Figure 7: Calls per second by status, and error rate (5 m window), HTTP on the left, gRPC on the right



Figure 8: p50 and p95 (5 m window), and in-flight calls; normal run: gRPC p50 25 ms, p95 220 ms (peak 975 ms during start-up); HTTP p50 7.5 ms, p95 87.5 ms



Figure 9: Slow run (`GRPC_SIMULATE_DELAY=2`): gRPC p50 1.81 s, p95 3.33 s; in-flight reaches 1

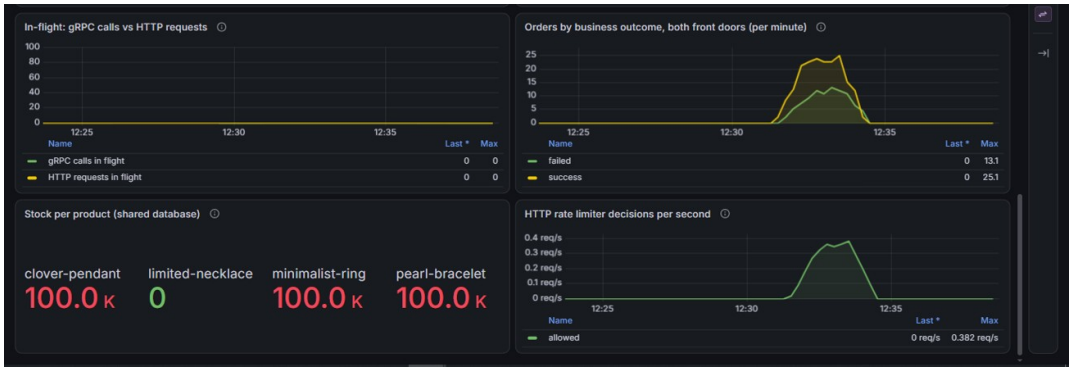


Figure 10: Stock per product (shared database) and rate-limiter decisions

Reading the panels honestly.

- Error rate counts every non-success call, including the deliberate failures in the traffic mix (about 45 to 50% here), so it shows the mix of the generator, not a faulty service.
- In-flight is 0 in the normal run because calls are sequential and take milliseconds. It only becomes visible in the slow run (Figure 9, maximum 1), which is why the slow switch exists.
- In the slow run the real handler time is about 2 s. The p95 of 3.33 s is higher because `histogram_quantile` interpolates inside the 2.5 s to 5 s bucket; the bucket edges limit the precision, not the service.
- The limiter panel only shows `allowed`, because the generator stays under 5 requests per 10 s. Rejections are demonstrated separately in section 2.6 and were counted by the same metric.
- Rate-limiter and stock panels are HTTP-side and shared-state panels, shown for completeness next to the gRPC ones.

5. Reproducing the results

```
pip install -r requirements-dev.txt
python -m pytest -q                          # 95 passed
docker compose up -d --build
python caller.py --scenario reliability   # Part A reliability evidence
python caller.py --scenario ratelimit     # Part A rate limit evidence
python demo_http_vs_grpc.py               # Part B success + failure, both
versions
python generate_traffic.py                # Part C traffic
```

Grafana http://localhost:3000, Prometheus http://localhost:9090, Kibana http://localhost:5601, Swagger UI http://localhost:8080. Full instructions and demo switches are in `README.md`.

6. Limitations

- The rate limiter is in memory and per process, and trusts `X-Forwarded-For`.
- SQLite with one writer at a time is fine for this scale, not for heavy write load.
- gRPC duration is server-side only; client-side latency is not measured, as the manual allows.
- gRPC traffic is plaintext inside the Docker network (no TLS).

7. Sources and reuse

- Course lecture notes, APIs lecture, section on the REST constraints (the manual's link points to 7.4; in my copy of the notes they are in 7.2); cancellation as a sub-resource (`POST /orders/123/cancellations`) is from the same lecture.
- gRPC workshop (`GRPC-WORKSHOP.md`), used as a general reference for the `.proto` and code-generation workflow.

- RFC 9457 (Problem Details for HTTP APIs), W3C Trace Context (`traceparent`), OpenAPI Specification 3.0.3, gRPC status codes documentation, Prometheus histogram and `histogram_quantile` documentation.
- Assignment 1 project (my own earlier work): Docker Compose stack for Prometheus, Grafana, Elasticsearch, Kibana, Filebeat and Node Exporter, and the JSON log format.